Necessity of Layout Transformations

Full Proof of Contribution C_1

(extracted from the pre-shortened paper draft)

This document contains the full theoretical proof underlying contribution C_1 of the SC26 paper "Why Schedule Transformations Are Not Enough: Layout Optimizations for Block-Granular I/O". The proof was cut from the paper's main body for space; the source is reproduced here verbatim from the pre-shortened April-1 manuscript snapshot.

1 When Are Layout Transformations Necessary?

While previous work focuses on schedule transformations to improve locality, we show that this recovery is not always possible. We establish the *necessity* of layout transformations in four steps: we define the memory and computational models (§1.1–§1.2); we construct blocked CDAGs on a concrete example to show how layout choices affect the I/O cost (§1.3); we then define a program model with layout mappings (§1.4); finally we connect layout mappings to blocked CDAGs (§1.5).

1.1 Memory Model

We adopt the external-memory model of Aggarwal and Vitter [1]: a processor operates on data in a fast memory of capacity S words, backed by an unbounded slow memory. Data moves between the two levels in *blocks* of B contiguous words. We use *block* throughout the formal model to refer to this unit of transfer; on current hardware, a block corresponds to a cache line. The I/O cost of an execution is the total number of block transfers; the minimum over all valid executions defines the I/O complexity $Q(G, S)$.

1.2 Computational Model

Computation DAG. A CDAG $G = (V, E)$ has vertices representing operations on data and directed edges representing dependencies: $(u, v) \in E$ means that the value at u is required to compute the value at v. Vertices with in-degree zero are *inputs*; vertices with out-degree zero are *outputs*.

Pebble games. The I/O cost of processing G is modeled by pebble games played on a CDAG [2, 3]. Pebbles are placed on and removed from vertices according to four rules: a vertex may be *loaded* from slow to fast memory, *stored* back, *computed* only when all its predecessors occupy fast memory, or *deleted*; at most S vertices may occupy fast memory at any time. A complete pebbling begins with all inputs in slow memory and ends with all outputs in slow memory. In the *red-blue* game of Hong and Kung [2], vertices may be recomputed arbitrarily; the *red-blue-white* variant [3] adds permanent white pebbles that forbid recomputation. The I/O cost is the number of load and store moves, and the minimum over all complete pebblings is $Q(G, S)$. The wavefront bounds used in this section are established in the recomputation-free (red-blue-white) model, which yields tighter lower bounds.

Quotient graph. Given an equivalence relation on the vertices of a CDAG, contracting each equivalence class (the group of elements that are equivalent under the relation - hereafter simply *class*) into a single node, yields a coarser graph that preserves only inter-class dependencies. We use this construction to derive I/O lower bounds.

Formally, let $\sim$ be an equivalence relation on V whose classes have size at most B. The *quotient graph* $G/\sim\ = (V/\sim,\ E/\sim)$ contracts each equivalence class $[v]$ into a single node:

$$V/\sim = \big\{\, [v] : v \in V \,\big\}, \tag{1}$$

$$E/\sim = \big\{\, ([u],[v]) : (u,v) \in E,\ [u] \neq [v] \,\big\}. \tag{2}$$

The condition $[u] \neq [v]$ excludes self-edges.

Wavefront bound. During the execution of a pebble game on a DAG, the *wavefront* at a given step is the set of vertices that must simultaneously reside in fast memory: vertices whose values have been computed but are still required by unprocessed successors [3, 4]. Formally, the wavefront $W_{\mathcal{R}}(v)$ under pebbling $\mathcal{R}$ is the set of vertices carrying a red pebble when v is computed. The width of the widest wavefront during execution lower-bounds the I/O cost: if there exist disjoint vertex sets $V_1, V_2 \subseteq V$ with m vertex-disjoint paths from V_1 to V_2, all paths must be simultaneously live at some point, giving $Q(G,S) \geq m - S$ [3].

Wavefront inflation. Let $\sim$ be an equivalence relation on V with classes of size B, and let V_1, V_2 be disjoint vertex sets in G with m vertex-disjoint paths from V_1 to V_2. Let $V_2' \subseteq V/\sim$ be the set of classes that contain vertices from V_2: a class $[v]_\sim$ belongs to V_2' if and only if $[v]_\sim \cap V_2 \neq \emptyset$. Define V_1' analogously.

Two nodes in $G/\sim$ are connected by an edge whenever at least one vertex in the first class shares an edge in G with at least one vertex in the second. A class in V_2' can therefore have up to B times as many in-neighbors in $G/\sim$ as any one of its constituent vertices has in G, since each of its B vertices may contribute edges from different classes. Each such in-neighbor—a class outside V_2' with an edge into V_2'—can independently anchor its own path from V_1' to V_2'.

If $|V/\sim| \geq B \cdot m \cdot \ell$, where ℓ is the maximum path length in $G/\sim$ (enough classes to route $B \cdot m$ paths of length ℓ without sharing any class), these paths can traverse mutually disjoint classes at every intermediate step, yielding $B \cdot m$ vertex-disjoint paths from V_1' to V_2' and inflating the wavefront by a factor of B. When the graph is too small for this condition to hold, the inflation factor is correspondingly less than B. We illustrate this inflation in §1.3.

1.3 Blocked CDAG Construction

We illustrate the computational model and the wavefront inflation bound on the elementwise matrix addition of Listing 1a with $N = 2$ and block size $B = 2$.

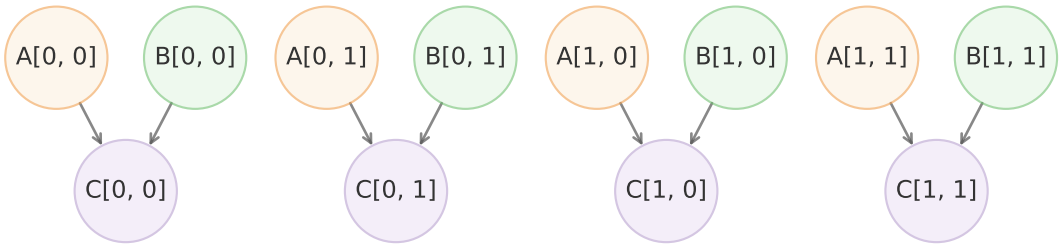
Original CDAG. Figure 1b shows the CDAG G: each output vertex $C[i,j]$ has exactly two predecessors, one from A and one from B.

Row-wise contraction. Contracting $A[i,0]$ with $A[i,1]$ (and likewise for B and C) groups elements that are consecutive under row-major storage into shared blocks. Each output-class node has exactly *two* input-class predecessors, yielding $m = 2$ vertex-disjoint paths from the input classes V_1 to the output classes V_2 (Figure 2a).

Column-wise contraction. Contracting $A[0,j]$ with $A[1,j]$ instead (grouping by column rather than row), while keeping the row-wise contraction for C, forces each output-class node to draw from *four* input-class predecessors, yielding $m = 4$ (Figure 2b). This is exactly the wavefront inflation of §1.2: the column-wise contraction fragments the optimal (row-wise) partition, doubling the number of vertex-disjoint paths and, by the wavefront bound, doubling the I/O lower bound.

```
for i in range(N):
    for j in range(N):
        C[i, j] = A[i, j] + B[i, j]
```

(a) Elementwise matrix addition.



(b) CDAG G (N=2) corresponding to Listing 1a. A: amber, B: green, C: lavender.

Figure 1: Running example: elementwise addition $C = A + B$ and its CDAG. Each output vertex has exactly two input predecessors.

(a) Quotient graph $G/\sim$ under row-wise contraction (B=2). Each output-class node has **2 input-class predecessors**; m=2 vertex-disjoint paths.

(b) Quotient graph $G/\sim$ under column-wise contraction for A, B and row-wise for C (B=2). Each output-class node has **4 input-class predecessors**; m=4 vertex-disjoint paths.

Figure 2: Quotient graphs for the running example (B=2). Dashed regions mark vertex sets V_1, V_2. Row-wise contraction (a) yields m=2; column-wise contraction (b) fragments the partition, doubling m to 4 and the minimum I/O with it.

1.4 Program Model

Following Kwasniewski et al. [5], we model a program as a collection of statements, each enclosed in a perfect loop nest (all statements reside at the innermost nesting level) of depth ℓ over an iteration domain $\mathcal{D} \subseteq \mathbb{N}^\ell$. Each statement accesses a set of arrays $\{A_0, \ldots, A_{M-1}\}$ through subscript functions $f_j \colon \mathbb{N}^\ell \to \mathbb{N}^{d_j}$ that map an iteration vector $\boldsymbol{\psi}$ to a logical index of A_j. A *schedule* π is a dependence-respecting total ordering of $\mathcal{D}$.

A *layout mapping* for array A_j is an injective function $\Phi_j \colon \mathbb{N}^{d_j} \to \mathbb{N}$ that maps each d_j-dimensional logical index to a unique physical memory address. Composing the subscript function with the layout mapping gives the physical address of a vertex v: $\Phi(v) = \Phi_{\mathrm{arr}(v)}(f_{\mathrm{arr}(v)}(\boldsymbol{\psi}_v))$, where $\mathrm{arr}(v)$ denotes the array accessed by v and $\boldsymbol{\psi}_v$ its iteration vector.

1.5 From Layout Mapping to Blocked CDAG

We now connect the program model to the computational model by showing that a layout mapping Φ induces a vertex partition on the CDAG, and the resulting quotient graph is exactly the blocked CDAG whose wavefront properties were established in §1.2.

Block address. The *block address* of a memory-accessing vertex v under layout mapping Φ is

$$\lambda_\Phi(v) = \lfloor \Phi(v) \,/\, B \rfloor. \tag{3}$$

The block address induces an equivalence relation on the vertices of G: two vertices u and v are *co-resident* if $\lambda_\Phi(u) = \lambda_\Phi(v)$, that is, they map to the same block under Φ. The *blocked CDAG* G_Φ is the quotient graph $G/{\sim_\Phi}$, where $u \sim_\Phi v$ iff $\lambda_\Phi(u) = \lambda_\Phi(v)$.

With this identification, the graph-theoretic results of §1.2 translate directly into statements about data layout.

Unit-granularity loads ($B = 1$). Every block contains a single element, so $\sim_\Phi$ places each vertex in its own class. The quotient graph $G_\Phi \cong G$ for any layout Φ, and the I/O cost is layout-independent.

Worst-case layout ($B > 1$). A layout that scatters co-accessed elements across different $\sim$-classes inflates the wavefront by up to a factor of B, yielding a proportional increase in the I/O lower bound that no schedule can eliminate. In the running example, row-major storage of all arrays yields the row-wise contraction with $m = 2$ (Figure 2a); storing A in row-major and B in column-major order doubles the wavefront to $m = 4$ due to the conflicting layouts (Figure 2b).

Compensation through fast memory. The factor-of-B gap can be closed by increasing fast-memory capacity from S to $B \cdot S$. When this capacity is unavailable, the inflation is unavoidable.

References

[1] H. Jia-Wei and H. T. Kung, "I/O Complexity: The Red-Blue Pebble Game," in *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing*, ser. STOC '81. New York, NY, USA: Association for Computing Machinery, 1981, p. 326–333. [Online]. Available: https://doi.org/10.1145/800076.802486

[2] A. Aggarwal and S. Vitter, Jeffrey, "The I/O Complexity of Sorting and Related Problems," *Commun. ACM*, vol. 31, no. 9, p. 1116–1127, Sep. 1988. [Online]. Available: https://doi.org/10.1145/48529.48535

[3] V. Elango, F. Rastello, L.-N. Pouchet, J. Ramanujam, and P. Sadayappan, "On characterizing the data movement complexity of computational dags for parallel execution," in *Proceedings*

of the 26th ACM Symposium on Parallelism in Algorithms and Architectures, ser. SPAA '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 296–306. [Online]. Available: https://doi.org/10.1145/2612669.2612694

[4] A. Olivry, J. Langou, L.-N. Pouchet, P. Sadayappan, and F. Rastello, "Automated derivation of parametric data movement lower bounds for affine programs," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 808–822. [Online]. Available: https://doi.org/10.1145/3385412.3385989

[5] G. Kwasniewski, T. Ben-Nun, L. Gianinazzi, A. Calotoiu, T. Schneider, A. N. Ziogas, M. Besta, and T. Hoefler, "Pebbles, graphs, and a pinch of combinatorics: Towards tight i/o lower bounds for statically analyzable programs," in *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '21. ACM, Jul. 2021, p. 328–339. [Online]. Available: http://dx.doi.org/10.1145/3409964.3461796